

Notebook: logistic_regression/logistic_regression_direct_implementation.ipynb

[--- MARKDOWN CELL ---]

Logistic Regression - Direct Implementation (Sklearn)

This notebook uses sklearn's LogisticRegression for comparison with our from-scratch implementation.

[--- CODE CELL ---]

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
```

[--- CODE CELL ---]

```
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.datasets import load_breast_cancer
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report, roc_curve, auc
```

[--- CODE CELL ---]

```
np.random.seed(42)
```

[--- MARKDOWN CELL ---]

Load and Prepare Data

[--- CODE CELL ---]

```
# Load Breast Cancer dataset
data = load_breast_cancer()
X = data.data
y = data.target

df = pd.DataFrame(X, columns=data.feature_names)
df['target'] = y
```

[--- CODE CELL ---]

```
df.head()
```

[--- CODE CELL ---]

```
df.describe()
```

[--- CODE CELL ---]

```
# Visualize class distribution
plt.figure(figsize=(8, 5))
sns.countplot(x='target', data=df)
plt.title("Class Distribution (0=Malignant, 1=Benign)")
plt.xlabel("Class")
plt.ylabel("Count")
plt.show()
```

[--- CODE CELL ---]

```
# Split and scale data
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
```

```
X_test_scaled = scaler.transform(X_test)
```

[--- MARKDOWN CELL ---]

Train Sklearn Logistic Regression

[--- CODE CELL ---]

```
# Create and train the model
lr = LogisticRegression(max_iter=1000, random_state=42)
lr.fit(X_train_scaled, y_train)

# Make predictions
y_pred = lr.predict(X_test_scaled)
y_pred_prob = lr.predict_proba(X_test_scaled)[: , 1]
```

[--- CODE CELL ---]

```
print("Coefficients shape:", lr.coef_.shape)
print("Intercept:", lr.intercept_[0])
```

[--- MARKDOWN CELL ---]

Evaluate the Model

[--- CODE CELL ---]

```
# Calculate accuracy
accuracy = accuracy_score(y_test, y_pred)
print(f"Accuracy (Sklearn): {accuracy:.4f}")
```

[--- CODE CELL ---]

```
# Classification Report
print("\nClassification Report:")
print(classification_report(y_test, y_pred, target_names=['Malignant', 'Benign']))
```

[--- CODE CELL ---]

```
# Plot confusion matrix
cm = confusion_matrix(y_test, y_pred)
plt.figure(figsize=(8, 6))
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues',
            xticklabels=['Malignant', 'Benign'],
            yticklabels=['Malignant', 'Benign'])
plt.title("Confusion Matrix - Sklearn Logistic Regression")
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.show()
```

[--- CODE CELL ---]

```
# Plot ROC Curve
fpr, tpr, thresholds = roc_curve(y_test, y_pred_prob)
roc_auc = auc(fpr, tpr)
plt.figure(figsize=(8, 6))
plt.plot(fpr, tpr, color='darkorange', lw=2, label=f'ROC curve (AUC = {roc_auc:.4f})')
plt.plot([0, 1], [0, 1], color='navy', lw=2, linestyle='--')
plt.xlim([0.0, 1.0])
plt.ylim([0.0, 1.05])
plt.xlabel('False Positive Rate')
plt.ylabel('True Positive Rate')
plt.title('ROC Curve - Sklearn Logistic Regression')
plt.legend(loc="lower right")
plt.grid(True, alpha=0.3)
```

```
plt.show()
```

[--- CODE CELL ---]

```
# Plot feature importance
```

```
feature_importance = np.abs(lr.coef_[0])
```

```
sorted_idx = np.argsort(feature_importance[::-1][:10])
```

```
plt.figure(figsize=(12, 6))
```

```
plt.barh(range(10), feature_importance[sorted_idx])
```

```
plt.yticks(range(10), [data.feature_names[i] for i in sorted_idx])
```

```
plt.xlabel("Absolute Coefficient Value")
```

```
plt.title("Top 10 Feature Importance (Sklearn Logistic Regression)")
```

```
plt.gca().invert_yaxis()
```

```
plt.tight_layout()
```

```
plt.show()
```